Chess Game — Phase 2

GUI Implementation Report

CS 3354 | TATENDA MACHIRORI

1. Project Overview

This report documents the Phase 2 implementation of the Chess Game project. Phase 2 focuses on building a fully functional Graphical User Interface (GUI) using the Java Swing framework. The application allows two players to play chess on a single machine, supports both click-to-move and drag-and-drop piece movement, handles captures, and declares a winner when a King is captured.

The project is structured using the Model-View-Controller (MVC) design pattern, separating game logic (model), display (view), and user interaction (controller) into distinct packages for maintainability and clarity.

2. Features Implemented

Core Requirements

Feature	Status	Description
8x8 Chessboard Display	**COMPLETE**	Alternating light/dark squares with rank and file labels (a–h, 1–8)
Piece Placement	**COMPLETE**	All 32 pieces rendered at standard starting positions using Unicode symbols
Click-to-Move	**COMPLETE**	Click a piece to select it (highlighted yellow), then click destination square
Drag and Drop	**COMPLETE**	Drag a piece with the mouse and drop it onto any destination square
Piece Capture	**COMPLETE**	Moving onto an opponent's piece removes it from the board permanently
Endgame Notification	**COMPLETE**	Capturing the King triggers a pop-up declaring the winner; game then exits

Extra GUI Features (3 implemented — minimum was 2)

Feature	Status	Points
Feature 1: Menu Bar with Game Controls (New Game, Save Game, Load Game, Quit)	**COMPLETE**	Required
Feature 2: Board Settings / Customization (4 color themes, 3 board sizes, coordinate toggle)	**COMPLETE**	**+5 Bonus**
Feature 3: Game History Panel + Undo Button (live move log, captured pieces, full undo)	**COMPLETE**	**+5 Bonus**

3. Code Architecture

The project is organized into three packages following the MVC pattern:

Package	Files	Responsibility
chess.model	Board, Piece, PieceType, PlayerColor, Move, BoardSettings	Game state, piece data, move history, undo logic, save/load settings
chess.view	BoardPanel, ChessFrame, HistoryPanel, SettingsDialog, PieceRenderer	All Swing GUI components: board rendering, windows, panels, piece drawing
chess.controller	GameController	Connects model to view; handles move execution, save/load, new game, undo

All classes include full Javadoc comments on every public method and constructor. The entry point is chess.Main, which launches the ChessFrame on the Swing Event Dispatch Thread using SwingUtilities.invokeLater().

4. GUI Screenshots

Screenshot 1 — Initial Game State

The screenshot below shows the chess application immediately after launch. All 32 pieces are placed at their standard starting positions: black pieces occupy ranks 7-8 (top) and white pieces occupy ranks 1-2 (bottom). The Game History panel on the right is empty, the Moves list is blank, and both Captured sections show no pieces. The turn indicator reads WHITE'S TURN.

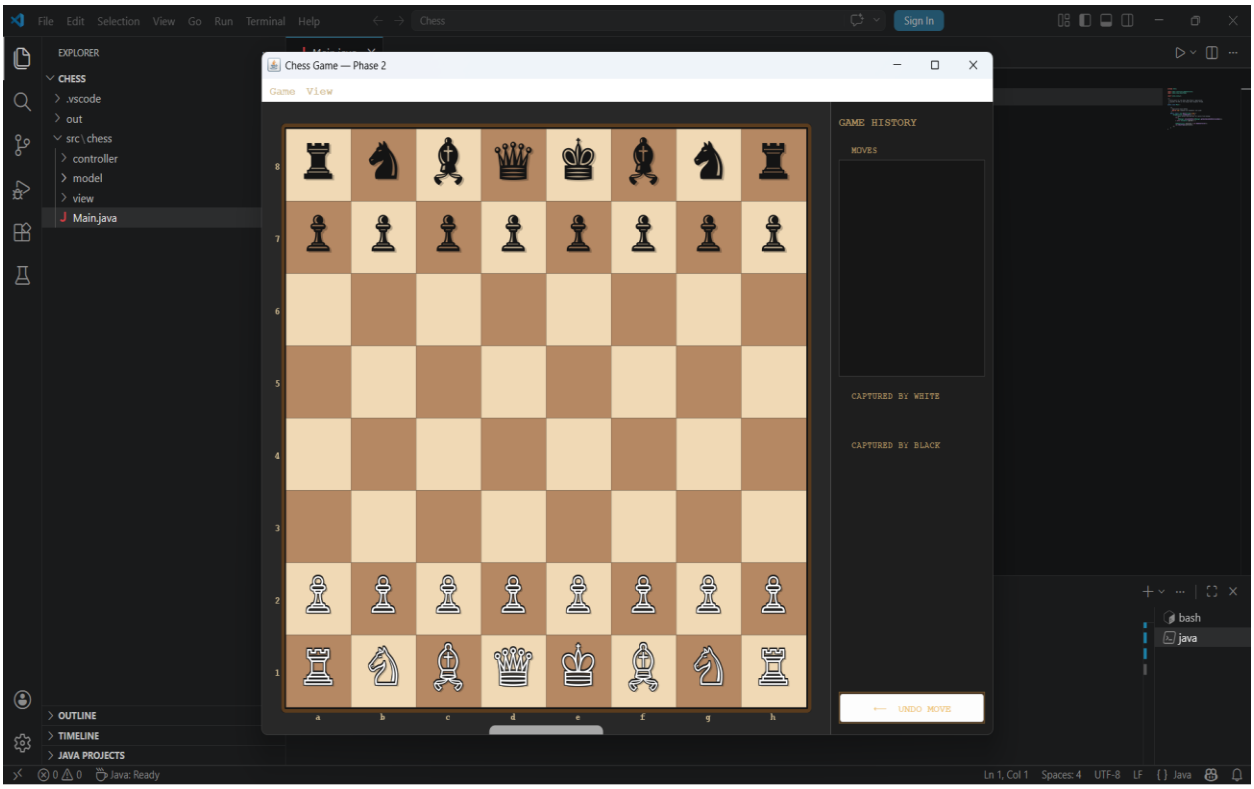

Figure 1: Chess game at startup — all 32 pieces at initial positions, no moves made.

Screenshots 2 & 3 — Mid-Game Captures and Winner Declaration

The screenshot below captures two required scenarios simultaneously. Several pieces have been moved and captured: the Game History panel shows 10 logged moves in algebraic notation, indicating each piece, start square, destination, and any capture. The Captured by White section shows a captured rook, and Captured by Black shows three captured pieces. A pop-up dialog declares 'Game Over — BLACK Wins!' after Black captured White's King, fulfilling the endgame notification requirement.

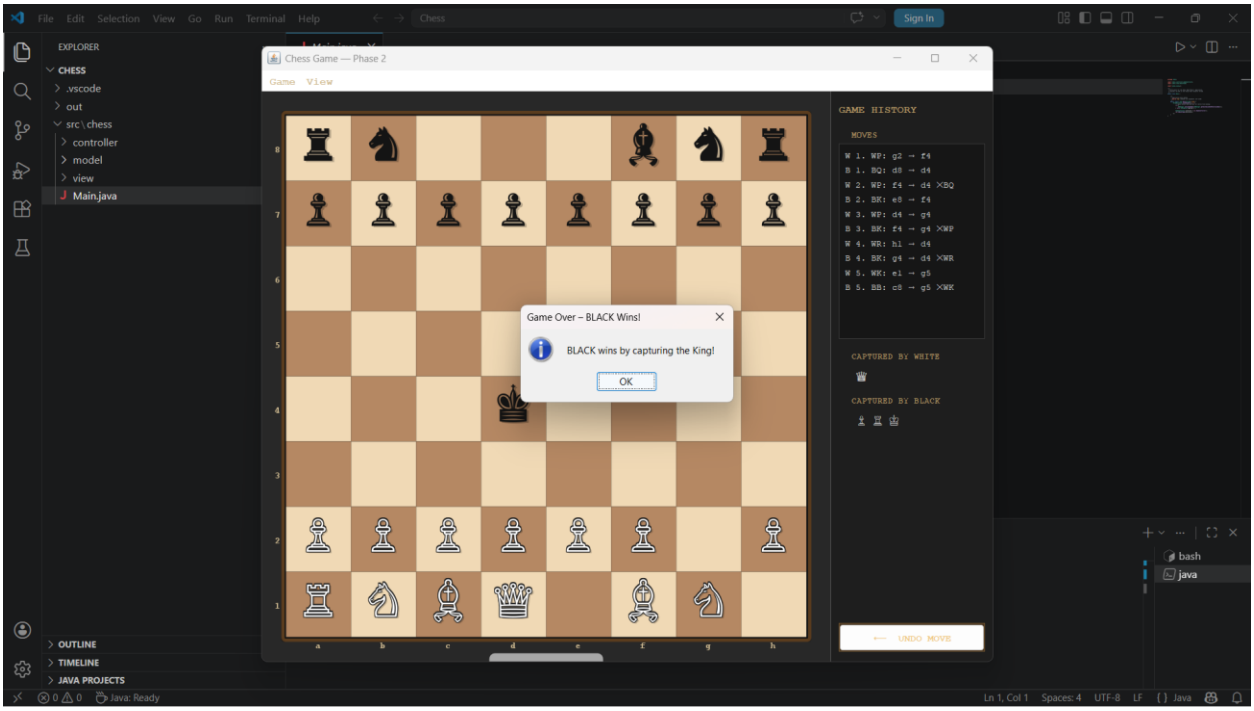

Figure 2 & 3: Mid-game with move history and captures shown, plus the King-capture winner popup.

5. Extra Feature Details

Feature 1: Menu Bar with Game Controls

A menu bar is implemented at the top of the window under the 'Game' menu with four items:

- **New Game (Ctrl+N):** Prompts the user for confirmation, then resets the board and all game data to the initial state.
- **Save Game (Ctrl+S):** Opens a file chooser dialog. The board state, move history, captured pieces, and settings are serialized to a .dat file using Java ObjectOutputStream.
- **Load Game (Ctrl+L):** Opens a file chooser dialog. A previously saved .dat file is deserialized and the board is fully restored to the exact state at the time of saving, including whose turn it was.
- **Quit (Ctrl+Q):** Exits the application immediately.

Feature 2: Board Settings / Customization Window

Accessible via the View menu, the Settings dialog allows players to customize board appearance in real time. Changes take effect immediately upon clicking Apply:

- **Board Theme:** Four themes: Classic Wood (tan/brown), Tournament Green (cream/green), Midnight Blue (light/dark blue), and Graphite (gray/dark gray).
- **Board Size:** Three sizes: Small (64px squares), Medium (80px squares), Large (96px squares). The window resizes dynamically.
- **Show Coordinates:** Toggle checkbox to show or hide the rank (1-8) and file (a-h) labels around the board edges.

Feature 3: Game History Panel with Undo Button

A persistent side panel on the right side of the window provides full game tracking:

- **Move History List:** Every move is logged in real time in the format: [color] [move#]. [piece]: [from] to [to]. Captures are indicated with an x symbol followed by the captured piece label.
- **Captured Pieces:** Two sections display Unicode chess symbols for all pieces captured by White and by Black respectively, updating after every move.
- **Undo Button:** Clicking Undo Move reverts the last move entirely — the piece returns to its origin square, any captured piece is restored, and the captured list updates. The button disables itself when no moves remain in history.

6. Notes and Comments

As specified in the Phase 2 requirements, move legality validation is intentionally not implemented — any piece can move to any unoccupied or opponent-occupied square. Full chess rule enforcement (check, checkmate, en passant, castling, pawn promotion) is outside the scope of this phase.

The application was developed and tested on Windows 11 using VS Code with JDK 25. Chess pieces are rendered using Unicode chess symbols (U+2654 to U+265F) with the Segoe UI Symbol font, available on all modern Windows systems. A drop shadow and outline effect is applied to improve piece visibility on both light and dark squares.

The GitHub repository contains all Phase 2 source files in the src/chess/ directory, separate from Phase 1 code. The project compiles and runs using standard javac and java commands with no external dependencies.